rails7/depot_o/app/controllers/orders_controller.rb

```
  def create
    @order = Order.new(order_params)
»   @order.add_line_items_from_cart(@cart)

    respond_to do |format|
      if @order.save
»       Cart.destroy(session[:cart_id])
»       session[:cart_id] = nil
»       format.html { redirect_to store_index_url, notice:
»         'Thank you for your order.' }
        format.json { render :show, status: :created,
          location: @order }
      else
        format.html { render :new, status: :unprocessable_entity }
        format.json { render json: @order.errors,
          status: :unprocessable_entity }
      end
    end
  end
```



Joe asks:

Aren’t You Creating Duplicate Orders?

Joe is concerned to see our controller creating `Order` model objects in two actions: `new` and `create`. He’s wondering why this doesn’t lead to duplicate orders in the database.

The answer is that the `new` action creates an `Order` object *in memory* simply to give the template code something to work with. Once the response is sent to the browser, that particular object gets abandoned, and it’ll eventually be reaped by Ruby’s garbage collector. It never gets close to the database.

The `create` action also creates an `Order` object, populating it from the form fields. This object *does* get saved in the database. So model objects perform two roles: they map data into and out of the database, but they’re also regular objects that hold business data. They affect the database only when you tell them to, typically by calling `save`.

We start by creating a new `Order` object and initialize it from the form data. The next line adds into this order the items that are already stored in the cart; we’ll write the method to do that in a minute.